

Candidate Submission

BUG REPORT

Task Mutation Authorization Audit

Investigation of the reported production issue — unauthorized task modification across project boundaries — including an audit of every task-mutation endpoint, root cause, fix, and regression coverage.

Prepared by

Youssef Bassiouny

Project

ProjectFlow — Full-Stack Engineer Assessment

Date

September 2026

CONTENTS

Bug Report: Task Mutation Authorization

Overview

Audit Summary

Bug 1 — Unauthorized Task Status Modification (Confirmed Vulnerability) — FIXED

Audited and Already Protected Endpoints

Regression Tests Added

Bug 2 — Task Number Race Condition (Previously Fixed)

TypeScript Errors Fixed (This Commit)

Verification Result

Bug Report: Task Mutation Authorization

Overview

This report documents the investigation into the reported production issue:

Some users appear to be able to modify tasks belonging to projects they are not members of.

All existing task mutation endpoints were audited against the actual source code.
Results distinguish **confirmed vulnerabilities** from **endpoints already protected**.

Audit Summary

Endpoint	Authenticated?	Resolves task?	Checks project access?	Verdict
<code>PATCH /tasks/:taskId</code>	✓ Yes	✓ Yes	✓ <code>assertCanView</code> + creator check	Already protected
<code>DELETE /tasks/:taskId</code>	✓ Yes	✓ Yes	✓ <code>assertCanManage</code>	Already protected
<code>PATCH /tasks/:taskId/status</code>	✓ Yes	✓ Yes	✗ None — missing	Confirmed vulnerable
<code>PATCH /tasks/:taskId/assignee</code>	✓ Yes	✓ Yes	✓ <code>assertCanView</code> + role check + assignee <code>canView</code>	Already protected
<code>GET /tasks/:taskId/activity</code>	✓ Yes	✓ Yes	✓ <code>assertCanView</code>	Already protected

Bug 1 — Unauthorized Task Status Modification (Confirmed Vulnerability) — FIXED

Description

Any authenticated user — including users who belong to a completely different organization — could change the status of **any task in the system** by calling `PATCH /tasks/:taskId/status` directly with a valid JWT. The frontend hides this control from unauthorized users, but the backend performed no authorization check, making the frontend restriction trivially bypassable via a direct API call.

Severity

HIGH — Broken Access Control / Insecure Direct Object Reference (OWASP A01)

Affected Code (before fix)

- `apps/api/src/tasks/tasks.controller.ts` — `updateStatus` handler
- `apps/api/src/tasks/tasks.service.ts` — `updateStatus` method

Root Cause

The original `updateStatus` controller handler did **not** extract `@CurrentUser('id')`, so no user identity was forwarded to the service. The service method accepted only `(taskId, dto)` and wrote the new status to the database without calling `ProjectAccessService` at all.

Vulnerable controller (before fix):

```
@Patch('tasks/:taskId/status')
updateStatus(
  @Param('taskId') taskId: string,
  @Body() dto: UpdateTaskStatusDto,
): Promise<TaskDetail> {
  return this.tasksService.updateStatus(toObjectId(taskId, 'task id'), dto);
  // no userId — authorization impossible
}
```

Vulnerable service (before fix):

```
async updateStatus(taskId: Types.ObjectId, dto: UpdateTaskStatusDto):
Promise<TaskDetail> {
  const task = await this.findTaskOrFail(taskId);
  // no projectAccessService call — any authenticated user proceeds
  task.status = dto.status;
  await task.save();
  return this.toDetail(task);
}
```

Steps to Reproduce (original vulnerability)

1. Register **User A** and **User B** (User B has no relationship to User A's project).
2. Authenticate as User A — create Project A, create Task A. Note the `taskId`.
3. Authenticate as User B (completely different organization).
4. `PATCH /tasks/<taskId>/status` with `Authorization: Bearer <User_B_JWT>` and body `{"status":"DONE"}`.
5. **Observed:** HTTP 200 OK — task status changed despite User B having no project membership.
6. **Expected:** HTTP 403 Forbidden — task unchanged.

Security / Business Impact

- Any authenticated user could change the status of any task across all organizations.
- Malicious actors could advance or regress any workflow state, disrupting project management.
- Frontend access controls were the only restriction — trivially bypassed via a direct API call.
- Affected all tasks in the system regardless of project or organization boundaries.

Fix Applied

Authorization level chosen: `assertCanView` — consistent with the `update` endpoint behavior. Status updates are a normal task operation for all project members; only project membership (not management permission) is required. The task's own `projectId` determines which project the check applies to — the client cannot supply an alternative project ID.

Controller fix — add `@CurrentUser('id') userId` and pass it to the service:

```
@Patch('tasks/:taskId/status')
updateStatus(
  @Param('taskId') taskId: string,
  @CurrentUser('id') userId: string,
  @Body() dto: UpdateTaskStatusDto,
): Promise<TaskDetail> {
  return this.tasksService.updateStatus(
    toObjectId(taskId, 'task id'),
    toObjectId(userId, 'user id'),
    dto,
  );
}
```

Service fix — add `userId` parameter and call `assertCanView` before mutating:

```
async updateStatus(
  taskId: Types.ObjectId,
  userId: Types.ObjectId,
  dto: UpdateTaskStatusDto,
): Promise<TaskDetail> {
  const task = await this.findTaskOrFail(taskId);
  const { project } = await this.projectAccessService.assertCanView(task.projectId, userId);
  task.status = dto.status;
  await task.save();
  return this.toDetail(task, project);
}
```

`assertCanView` throws `ForbiddenException` if the user is neither an organization OWNER/ADMIN nor an explicit project member. Authorization uses the task's own stored `projectId` — never a client-supplied value.

Audited and Already Protected Endpoints

`PATCH /tasks/:taskId` — update

- Extracts `@CurrentUser('id') userId` in the controller.
- Service calls `assertCanView(task.projectId, userId)` first.
- Secondary check: only project managers / org OWNER/ADMIN / the task creator may edit task fields.
- Regular project members who did not create the task receive `403`.
- **No vulnerability found.**

DELETE /tasks/:taskId — remove

- Extracts `@CurrentUser('id')` `userId` in the controller.
- Service calls `assertCanManage(task.projectId, userId)` — correctly requires elevated permission.
- Regular project members (MEMBER role) cannot delete tasks; only PROJECT_MANAGER and org OWNER/ADMIN can.
- **No vulnerability found.**

PATCH /tasks/:taskId/assignee — assignTask

- Extracts `@CurrentUser('id')` `userId` in the controller.
- Service calls `assertCanView(task.projectId, actorId)` — verifies project membership first.
- Secondary check: `canManage(access)` required to assign another member; regular MEMBERS can only assign themselves.
- Verifies the assignee is a project member via `ProjectMembersService.findRole`.
- **No vulnerability found.**

GET /tasks/:taskId/activity — findActivity

- Extracts `@CurrentUser('id')` `userId` in the controller.
- Service calls `assertCanView(task.projectId, userId)` before returning any activity records.
- **No vulnerability found.**

Regression Tests Added

File: `apps/api/test/tasks.e2e.spec.ts` — `describe('task mutation authorization')`

Test	Covers
Authorized member can update a task title	Happy path — <code>PATCH /tasks/:taskId</code>
Outsider receives 403 on update; task unchanged	Cross-project unauthorized <code>PATCH /tasks/:taskId</code>
Outsider receives 403 on delete; task still exists	Cross-project unauthorized <code>DELETE /tasks/:taskId</code>
Authorized member can change task status	Happy path — <code>PATCH /tasks/:taskId/status</code>
Outsider receives 403 on status change; status unchanged	Core regression for confirmed vulnerability
Owner (OWNER org role) can delete a task; task then 404s	Confirms org-level OWNER access still works

Additional coverage in `describe('task assignment')` and `describe('task assignment activity')`:

- Project member can assign themselves
- PROJECT_MANAGER can assign any project member
- Org OWNER can assign any project member
- Regular MEMBER cannot assign another member (403)
- Non-project user cannot be assigned (403)
- Unauthorized actor cannot call assign endpoint (403)
- Activity records created and isolated per-task
- Cursor-based pagination of activity history

Bug 2 — Task Number Race Condition (Previously Fixed)

This is a separate data integrity issue. The concurrency fix (atomic `taskSequenceModel` counter using `$inc`) was implemented before this audit and is covered by the existing concurrent-creation test in `tasks.e2e.spec.ts`.

TypeScript Errors Fixed (This Commit)

During this audit, three pre-existing TypeScript type errors introduced with the activity API implementation were identified and resolved:

File	Error	Fix
<code>task-activity.schema.ts</code>	<code>AssigneeChangedMetadata</code> not assignable to <code>Record<string, unknown></code>	Added index signature to interface
<code>tasks.service.ts</code>	Mongoose <code>.where().lt()</code> chain rejects <code>Date / ObjectId</code> types	Replaced with plain <code>\$lt</code> filter object
<code>tasks.service.ts</code>	<code>items[items.length - 1]</code> possibly <code>undefined</code>	Added <code>!</code> assertion (guarded by <code>items.length > 0</code> check)

Verification Result

After applying all fixes:

Check	Result
<code>pnpm typecheck</code>	✓ Pass
<code>pnpm lint</code>	✓ Pass
<code>pnpm test</code>	✓ Pass (all suites)
<code>pnpm build</code>	✓ Pass